K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

# Basics of Neural Network & Deep Learning Lab ENCA354
# BCA(AI & DS)

# LAB FILE


**SUBMITTED BY:**

Vaishnavi Pushp

2301201126

**SUBMITTED TO:**

Mr. Anoja Mishra

# INDEX


| SR. NO. | EXPERIMENTS | PAGE NO. | TEACHER SIGN |
|---|---|---|---|
| 01. | **Neural Network using TensorFlow (Low-Level API)** | 3 | |
| 02. | **Neural Network using basic python (from scratch)** | 4 | |
| 03. | **Neural Network using Keras** | 5-6 | |
| 04. | **Agricultural Price Prediction (Regression Problem)** | 7-9 | |
| 05. | **MNIST Digital Classification using ANN** | 10-11 | |
| 06. | **Plant Disease Detection using CNN** | 12-14 | |

# 1. Neural Network using TensorFlow (Low-Level API)

- **Build a simple feedforward neural network using TensorFlow 1.x style (placeholders, sessions)**
- **Understand computation graph and manual weight initialization**

```python
import tensorflow as tf
import numpy as np
```

✓ 42.2s Python

```
C:\Users\acer\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python31
  if not hasattr(np, "object"):
```

```python
np.random.seed(42)
X_data = np.random.rand(100, 2).astype(np.float32)
y_data = (X_data[:, 0] + X_data[:, 1] > 1).astype(np.float32).reshape(-1, 1)
```

✓ 0.0s Python

```python
tf.compat.v1.reset_default_graph()
tf.compat.v1.disable_eager_execution()
```

✓ 1.0s Python

```
WARNING:tensorflow:From C:\Users\acer\AppData\Local\Temp\ipykernel_20068\775658710.py:1: The name tf.reset_default_graph i
```

```python
X = tf.compat.v1.placeholder(tf.float32, shape=[None, 2], name='X')
y = tf.compat.v1.placeholder(tf.float32, shape=[None, 1], name='y')

W1 = tf.Variable(tf.random.normal([2, 4]), name='W1')
b1 = tf.Variable(tf.zeros([4]), name='b1')
W2 = tf.Variable(tf.random.normal([4, 1]), name='W2')
b2 = tf.Variable(tf.zeros([1]), name='b2')
```

✓ 0.0s Python

```python
hidden = tf.nn.relu(tf.matmul(X, W1) + b1)
output = tf.nn.sigmoid(tf.matmul(hidden, W2) + b2)

loss = tf.reduce_mean(tf.square(output - y))
optimizer = tf.compat.v1.train.GradientDescentOptimizer(0.01).minimize(loss)
```

✓ 0.0s Python

```
WARNING:tensorflow:From C:\Users\acer\AppData\Local\Temp\ipykernel_20068\1933492293.py:5: The name tf.train.GradientDescer
```

```python
with tf.compat.v1.Session() as sess:
    sess.run(tf.compat.v1.global_variables_initializer())
    for epoch in range(500):
        _, l = sess.run([optimizer, loss], feed_dict={X: X_data, y: y_data})
        if (epoch + 1) % 100 == 0:
            print(f'Epoch {epoch+1}, Loss: {l:.4f}')
    preds = sess.run(output, feed_dict={X: X_data})
    acc = np.mean((preds > 0.5).astype(int).flatten() == y_data.flatten())
    print(f'Accuracy: {acc*100:.2f}%')
```

✓ 0.2s Python

```
Epoch 100, Loss: 0.3936
Epoch 200, Loss: 0.3806
Epoch 300, Loss: 0.3610
Epoch 400, Loss: 0.3336
Epoch 500, Loss: 0.3022
Accuracy: 56.00%
```

# 2. Neural Network using Basic Python (From Scratch)

a. Implement a neural network using **NumPy only**
b. Include:
   i. Forward propagation
   ii. Backpropagation
   iii. Gradient descent

```python
import numpy as np
```
✓ 1.0s Python

```python
def sigmoid(x): return 1 / (1 + np.exp(-x))
def sigmoid_deriv(x): return x * (1 - x)

X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
y = np.array([[0],[1],[1],[0]], dtype=float)
```
✓ 0.0s Python

```python
np.random.seed(42)
W1 = np.random.randn(2, 4); b1 = np.zeros((1, 4))
W2 = np.random.randn(4, 1); b2 = np.zeros((1, 1))
lr = 0.1
```

```python
for epoch in range(10000):
    # Forward pass
    a1 = sigmoid(X.dot(W1) + b1)
    a2 = sigmoid(a1.dot(W2) + b2)
    loss = np.mean((y - a2) ** 2)

    # Backpropagation
    d2 = -(y - a2) * sigmoid_deriv(a2)
    W2 -= lr * a1.T.dot(d2);  b2 -= lr * d2.sum(axis=0, keepdims=True)
    d1 = d2.dot(W2.T) * sigmoid_deriv(a1)
    W1 -= lr * X.T.dot(d1);   b1 -= lr * d1.sum(axis=0, keepdims=True)

    if (epoch + 1) % 2000 == 0:
        print(f'Epoch {epoch+1}, Loss: {loss:.4f}')

print('\nPredictions:', np.round(a2).flatten())
```
✓ 0.3s Python

Output:

```
Epoch 2000, Loss: 0.2123
Epoch 4000, Loss: 0.0571
Epoch 6000, Loss: 0.0107
Epoch 8000, Loss: 0.0047
Epoch 10000, Loss: 0.0028

Predictions: [0. 1. 1. 0.]
```

# 3. Neural Network using Keras

a. Build a model using **Keras Sequential API**
b. Train on a small dataset (e.g., Iris or dummy data)
c. Visualize accuracy and loss

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelBinarizer, StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
iris = load_iris()
X = StandardScaler().fit_transform(iris.data)
y = LabelBinarizer().fit_transform(iris.target)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = Sequential([
    Dense(16, activation='relu', input_shape=(4,)),
    Dense(8,  activation='relu'),
    Dense(3,  activation='softmax')
])
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.summary()

history = model.fit(X_train, y_train, epochs=100, batch_size=16,
                    validation_data=(X_test, y_test), verbose=1)

loss, acc = model.evaluate(X_test, y_test, verbose=0)
print(f'Test Accuracy: {acc*100:.2f}%')
```

```
Model: "sequential"
```

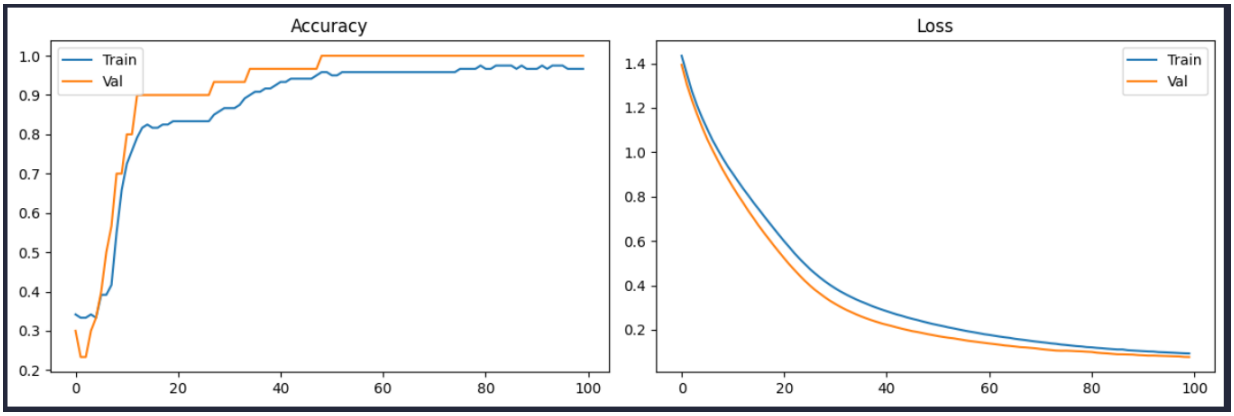
| Layer (type) | Output Shape | Param # |
|---|---|---|
| dense (Dense) | (None, 16) | 80 |
| dense_1 (Dense) | (None, 8) | 136 |
| dense_2 (Dense) | (None, 3) | 27 |

```
Total params: 243 (972.00 B)

Trainable params: 243 (972.00 B)

Non-trainable params: 0 (0.00 B)
```

```python
plt.figure(figsize=(12, 4))
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Val')
plt.title('Accuracy'); plt.legend()
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train')
plt.plot(history.history['val_loss'], label='Val')
plt.title('Loss'); plt.legend()
plt.tight_layout(); plt.show()
```



# 4. Agriculture Price Prediction (Regression Problem)

a. Take any dataset from GitHub (e.g., crop price dataset)
b. Build a regression model using ANN
c. Evaluate using:
   i. MSE
   ii. $R^2$ Score

```python
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

Python

```python
df = pd.read_csv("dataset_indian_crop_price.csv")

print(df.head())
print(df.info())
```

Python

```
   Unnamed: 0     Date        State    City Crop Type  Season  \
0           0  2023-01  Maharashtra  Mumbai     Wheat  Kharif
1           1  2023-02  Maharashtra  Mumbai     Wheat  Kharif
2           2  2023-03  Maharashtra  Mumbai     Wheat  Kharif
3           3  2023-04  Maharashtra  Mumbai     Wheat  Kharif
4           4  2023-05  Maharashtra  Mumbai     Wheat  Kharif

   Temperature (°C)  Rainfall (mm)  Supply Volume (tons)  \
0              37.1          204.5               3583.93
1              21.2           57.8               2944.07
2              36.3          129.4               3956.04
3              19.5           24.3               1651.50
4              36.6          218.2               2173.52

   Demand Volume (tons)  Transportation Cost (₹/ton)  \
0               1345.60                       302.65
1               1698.04                       393.07
2               2209.66                       350.68
3                576.15                       274.45
4               1185.90                       382.61

   Fertilizer Usage (kg/hectare)  Pest Infestation (0-1)  Market Competition  \
0                         219.62                    0.12                0.72
1                         263.00                    0.38                0.05
```

```python
df = df.dropna()
```

Python

```python
le = LabelEncoder()

for col in df.select_dtypes(include=['object']).columns:
    df[col] = le.fit_transform(df[col])

X = df.drop("Price", axis=1)
y = df["Price"]
```

Python

+ Code + Markdown

```python
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Python

```python
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Python

```python
model = Sequential()

# Input Layer + Hidden Layer
model.add(Dense(64, activation='relu', input_shape=(X_train.shape[1],)))

# Hidden Layer
model.add(Dense(32, activation='relu'))

# Output Layer (Regression → 1 neuron, no activation)
model.add(Dense(1))
```

Python

```
C:\Users\acer\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python31
```

```python
model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)
```

Python

```python
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2
)
```

Python

```python
y_pred = model.predict(X_test)
```

Python

```
300/300 ━━━━━━━━━━━━━━━━━━━━ 0s 623us/step
```

```python
mse = mean_squared_error(y_test, y_pred)
print("MSE:", mse)
```

Python

```
MSE: 0.15609756208485526
```

```python
r2 = r2_score(y_test, y_pred)
print("R² Score:", r2)
```

Python

```
R² Score: 0.9999769865501091
```

# 5. MNIST Digit Classification using ANN

a. Load MNIST dataset
b. Build a simple ANN (fully connected network)
c. Evaluate accuracy and confusion matrix

```python
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.utils import to_categorical
from sklearn.metrics import confusion_matrix
```

✓ 24.7s Python

```python
(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train, X_test = X_train / 255.0, X_test / 255.0
y_train_cat = to_categorical(y_train, 10)
y_test_cat  = to_categorical(y_test, 10)
```

✓ 0.3s Python

```python
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(256, activation='relu'),
    Dense(128, activation='relu'),
    Dense(64,  activation='relu'),
    Dense(10,  activation='softmax')
])
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.summary()
```

✓ 0.1s Python

| Layer (type) | Output Shape | Param # |
|---|---|---|
| flatten (Flatten) | (None, 784) | 0 |
| dense (Dense) | (None, 256) | 200,960 |
| dense_1 (Dense) | (None, 128) | 32,896 |
| dense_2 (Dense) | (None, 64) | 8,256 |
| dense_3 (Dense) | (None, 10) | 650 |

Total params: 242,762 (948.29 KB)

Trainable params: 242,762 (948.29 KB)

Non-trainable params: 0 (0.00 B)

```python
history = model.fit(X_train, y_train_cat, epochs=15,
                    batch_size=128, validation_data=(X_test, y_test_cat))
```

✓ 21.8s Python

```python
loss, acc = model.evaluate(X_test, y_test_cat, verbose=0)
print(f'Test Accuracy: {acc*100:.2f}%')
```

✓ 0.4s Python

```
Test Accuracy: 97.66%
```

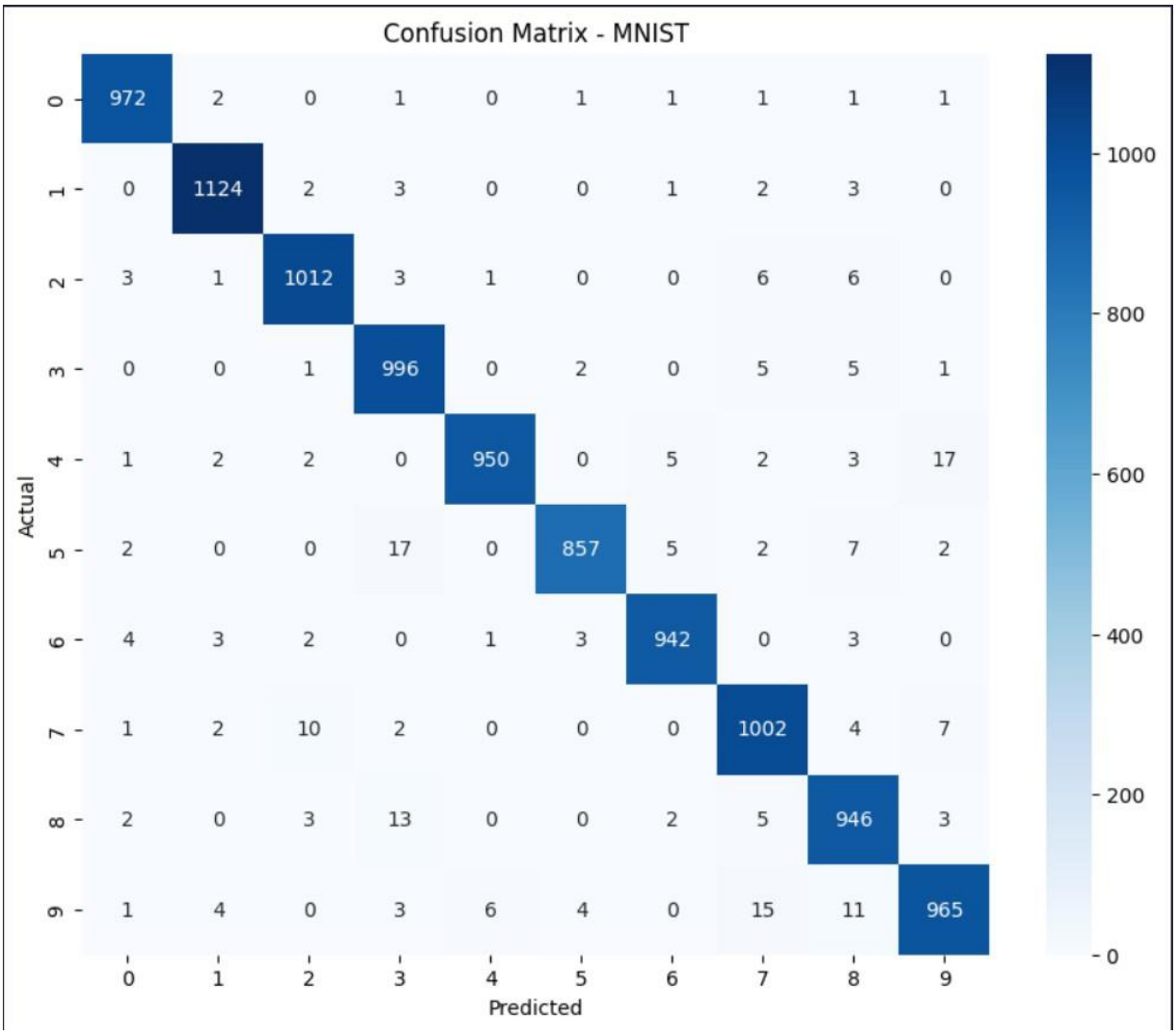
```python
y_pred = np.argmax(model.predict(X_test), axis=1)
cm = confusion_matrix(y_test, y_pred)
```

✓ 0.4s Python

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 874us/step
```

```python
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=range(10), yticklabels=range(10))
plt.xlabel('Predicted'); plt.ylabel('Actual')
plt.title('Confusion Matrix - MNIST'); plt.show()
```

✓ 0.2s Python



# 6. Plant Disease Detection using CNN

a. Use plant leaf dataset (e.g., from Kaggle)
b. Multi-class classification of diseases
c. Save basic prediction model

```python
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

✓ 36.1s Python

```python
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True
)

val_test_datagen = ImageDataGenerator(rescale=1./255)
```

✓ 0.0s Python

```python
train_data = train_datagen.flow_from_directory(
    r'C:\Users\acer\OneDrive\Desktop\College Work\SEM 6\Neural Networks and Deep Learning\lab projects\archive\Potato\
    target_size=(128,128),
    batch_size=32,
    class_mode='categorical'
)

val_data = val_test_datagen.flow_from_directory(
    r'C:\Users\acer\OneDrive\Desktop\College Work\SEM 6\Neural Networks and Deep Learning\lab projects\archive\Potato\
    target_size=(128,128),
    batch_size=32,
    class_mode='categorical'
)

test_data = val_test_datagen.flow_from_directory(
    r'C:\Users\acer\OneDrive\Desktop\College Work\SEM 6\Neural Networks and Deep Learning\lab projects\archive\Potato\
    target_size=(128,128),
    batch_size=32,
    class_mode='categorical',
    shuffle=False
)
```

✓ 0.0s Python

```
Found 900 images belonging to 3 classes.
Found 300 images belonging to 3 classes.
Found 300 images belonging to 3 classes.
```

```python
model = Sequential()

# Conv Layer 1
model.add(Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)))
model.add(MaxPooling2D(2,2))

# Conv Layer 2
model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D(2,2))

# Conv Layer 3
model.add(Conv2D(128, (3,3), activation='relu'))
model.add(MaxPooling2D(2,2))

# Flatten
model.add(Flatten())

# Dense Layer
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))

# Output Layer
model.add(Dense(train_data.num_classes, activation='softmax'))
```

✓ 0.0s Python

```python
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

✓ 0.0s Python

```python
history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=10
)
```

✓ 1m 3.3s Python

```python
loss, accuracy = model.evaluate(test_data)
print("Test Accuracy:", accuracy)
```

✓ 2.9s Python

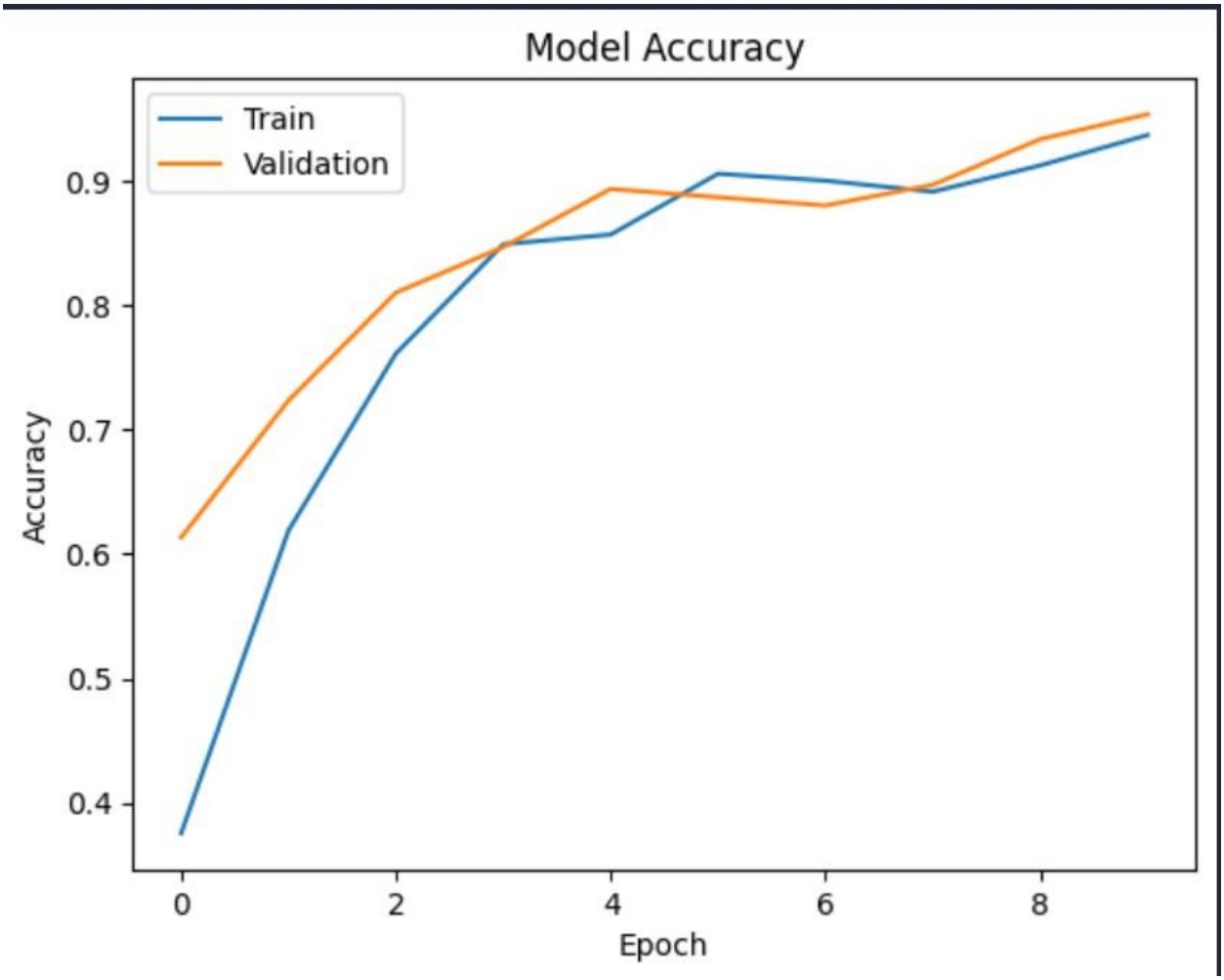
```
10/10 ━━━━━━━━━━━━━━━━━━━━ 2s 260ms/step - accuracy: 0.9633 - loss: 0.0914
Test Accuracy: 0.9633333086967468
```

```python
model.save("potato_disease_model.h5")
```

✓ 0.0s Python

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file f
```

```python
import matplotlib.pyplot as plt

plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'])
plt.show()
```

✓ 0.1s Python